

Tarea 1 2026

Tópicos en Grandes Volúmenes de Datos

Count-Min Sketch y CountSketch en ventanas deslizantes para detección de ataques

Grupo: Sebastián Navarrete Caro
Profesora: Cecilia Hernández
Ayudante: Álvaro Guzmán

1. Introducción

Los sketches permiten resumir un flujo continuo de datos en una estructura de tamaño fijo (y usualmente pequeño), a partir de la cual se pueden estimar propiedades del flujo, como la frecuencia de cada elemento, sin necesidad de almacenarlo completo. Esta tarea tiene como objetivo utilizar la propiedad de linealidad de dos sketches de frecuencia, Count-Min Sketch (CMS) y CountSketch (CS), para implementar una ventana deslizante que permita detectar cambios anómalos en el tráfico de red producidos por dos ataques: un DDoS (*Distributed Denial of Service*), que busca interrumpir el funcionamiento de un servidor saturándolo con tráfico enviado desde muchas fuentes, y un *scan*, en el que una única fuente contacta a una gran cantidad de destinos para descubrir qué máquinas y servicios están activos. Por esta diferencia, el DDoS se analiza según la IP de destino (la víctima) y el scan según la IP de origen (el atacante).

Para lograr esto se utilizó una traza real de 15 minutos del MAWI Working Group Traffic Archive¹, sobre la cual se inyectaron ataques sintéticos mediante la herramienta `inject_attack.py`, entregada para este trabajo.

Para evaluar estas anomalías se utiliza el concepto de *heavy hitter* (HH). Un HH es una IP (de origen o de destino, según el ataque) que concentra al menos el 1 % de los paquetes de una ventana, es decir, cuya frecuencia $f_j(x)$ cumple $f_j(x) \geq \lceil \phi N_j \rceil$, con $\phi = 0,01$ y N_j el total de paquetes de la ventana j . En este trabajo se usa una ventana $W = 60$ s dividida en 6 subventanas de 10s, de modo que la ventana avanza y se evalúa cada 10s. Un ataque se considera detectado en la primera ventana en que la IP involucrada es clasificada como heavy hitter.

2. Método

2.1. Ventana deslizante

La traza se recorre solamente una vez, en orden temporal. Sea t_0 el tiempo del primer paquete y t_e el del último. El tiempo se divide en subventanas de $p = 10$ s, donde la subventana q cubre el intervalo $(t_0 + (q - 1)p, t_0 + qp]$. La ventana activa está formada por las últimas $m = W/p = 6$ subventanas ($W = 60$ s) y se evalúa cada p segundos, en $\tau_k = t_0 + W + kp$, para $k = 0, 1, \dots$ mientras $\tau_k \leq t_e$ (para la traza utilizada se tienen 84 evaluaciones).

Cada subventana tiene su propio sketch S_q , guardado en un anillo de $m = 6$ posiciones, las cuales se denominarán ranuras, que se reutilizan de forma circular. La subventana q ocupa la ranura $(q - 1) \bmod 6$ (Figura 1). El sketch de la ventana se mantiene en un sketch agregado A_k , igual a la suma de los sub-sketches vigentes. En la primera evaluación ($k = 0$) se precargan las seis primeras subventanas, es decir, los paquetes con tiempo en $(t_0, t_0 + 60]$. En cada evaluación siguiente, primero se expira la subventana más antigua, restando su sketch del agregado y limpiando su ranura, y después se cargan en esa misma ranura los paquetes de la nueva subventana:

$$A_0 = \sum_{q=1}^6 S_q, \quad A_k = A_{k-1} - S_k + S_{k+6} \quad (k \geq 1). \quad (1)$$

¹<https://mawi.wide.ad.jp/mawi/samplepoint-F/2018/201812031400.pcap.gz>

Así, entre dos evaluaciones consecutivas solo cambia una ranura, y ningún paquete se procesa más de una vez.

Junto a los sketches se mantiene el total exacto de paquetes de la ventana, N_k , con un anillo análogo de seis contadores, que permite definir el umbral $T_k = \lceil \phi N_k \rceil$. Este N_k resultó idéntico a la columna N de `exact_hh` en las 84 ventanas, tanto en la traza sin ataques como en las 30 corridas con ataque (comprobación incluida en `actividad1_codigo.py` y `tabla_resumen.py`), lo que verifica que las subventanas están alineadas con las de la referencia exacta.

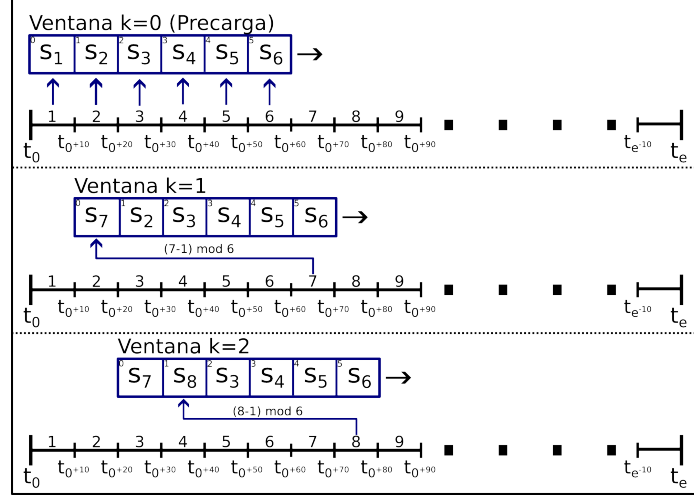


Figura 1: Subventanas de 10s y anillo de seis sub-sketches en las tres primeras evaluaciones. Cada caja muestra el contenido de las ranuras en memoria, no su orden temporal: en $k = 1$, S_7 reemplaza a S_1 en la misma ranura.

2.2. Sketches

Para cada ventana se mantienen en realidad dos sketches en paralelo, uno CMS y otro CS, cada uno con su propio anillo de sub-sketches y su agregado. Ambos estiman la frecuencia de una clave x (la IP de origen o de destino, según el ataque) a partir de una matriz C de $d \times w$ contadores, donde cada una de las d filas tiene su propia función hash h_j , que asigna a cada clave una de las w columnas. Como hay muchas más claves que columnas, claves distintas comparten celdas (colisiones) y sus paquetes se acumulan en el mismo contador. Al reducir w , cada celda recibe más claves y el error de la estimación aumenta. CMS y CS se diferencian en cómo se actualiza la matriz al observar una clave y en cómo se estima su frecuencia.

En CMS, una actualización (x, c) suma c en la celda $C[j, h_j(x)]$ de cada fila. En este trabajo cada paquete es una actualización con $c = 1$. En CS la actualización es la misma, pero se suma $s_j(x)c$, donde $s_j(x) \in \{-1, +1\}$ es el signo de la clave x en la fila j , obtenido con una segunda función hash, por lo que es siempre el mismo para una clave dada. Para estimar la frecuencia de x , CMS toma el mínimo de las d celdas, y CS la mediana de las d celdas multiplicadas por su signo $s_j(x)$, lo que deshace el signo aplicado al insertar:

$$\hat{f}_{\text{CMS}}(x) = \min_j C[j, h_j(x)], \quad \hat{f}_{\text{CS}}(x) = \text{mediana}_j s_j(x) C[j, h_j(x)]. \quad (2)$$

Como en CMS las colisiones solo suman, su estimación nunca es menor que la frecuencia real, y el mínimo elige la fila menos afectada por colisiones. En CS las colisiones suman o restan según el signo, de modo que el error puede ser positivo o negativo. Si la estimación resulta negativa se trunca a 0 al reportarla, ya que una frecuencia no puede ser negativa (los contadores internos no se truncan).

Las funciones hash se implementaron con MurmurHash3 (32 bits), utilizada en las ayudantías del curso. Recibe la IP y una semilla, y la columna se obtiene como su resultado módulo w . Cada

fila usa una semilla distinta, y el signo de CS usa otras d semillas, todas generadas a partir de una semilla del experimento. Las mismas funciones se comparten en todas las matrices de un mismo tipo de sketch, condición necesaria para que la suma y resta de la ecuación (1) sean válidas.

2.3. Cambio de frecuencia

Además de la frecuencia, se estima el cambio de frecuencia entre ventanas consecutivas, $\Delta f_k(x) = f_k(x) - f_{k-1}(x)$. Por linealidad, su sketch es la diferencia entre la subventana que entra y la que sale, $\Delta A_k = A_k - A_{k-1} = S_{k+6} - S_k$, que se obtiene guardando una copia de la subventana saliente antes de limpiar su ranura. Como ΔA_k puede tener valores negativos, en CMS no se usa el mínimo sino la mediana de las d celdas (CMS-mediana), mientras que CS usa su estimador habitual (ecuación (2)). En ambos casos el resultado no se trunca.

3. Resultados

En todos los experimentos se usó $d = 5$ y $w \in \{256, 1024, 4096\}$. En la traza sin ataques las funciones hash se generaron con una sola semilla, y en los ataques los resultados se evaluaron con cinco semillas distintas.

3.1. Validación en la traza sin ataques

Se validaron diez claves de la traza sin ataques, cinco IP de destino y cinco de origen. Estas se eligieron a partir del ranking de frecuencia (top- k) que entrega `exact_hh` para la primera ventana ($k = 0$), tomando las claves en las posiciones 1, 10, 50, 100 y 250. Para cada clave se comparó la estimación de cada sketch con la frecuencia exacta de `exact_hh` en las 84 ventanas. Las Tablas 3 y 4 del Anexo muestran las claves de destino y de origen, con la frecuencia exacta media entre ventanas, y el error absoluto y relativo como media $\pm$ desviación estándar.

Para las diez claves y los tres anchos w , se observa que el error absoluto de CMS es mayor que el de CS en todos los casos. Esto es esperable, ya que en CMS las colisiones siempre suman a la celda de la clave, mientras que en CS suman o restan según el signo y tienden a compensarse. En general, el error absoluto es parecido para las distintas claves con un mismo ancho w , exceptuando con CS y $w = 256$ las IP 204.93.236.31 y 163.210.11.14, cuyo error es menor. Con $w = 256$ el error de CMS está en el orden de los miles de paquetes, y al aumentar w a 4096 se reducen las colisiones y pasa al orden de los cientos. Como el error absoluto es casi el mismo para claves grandes y pequeñas, el error relativo depende del tamaño de la clave. Para el heavy hitter, un error de 8.655 paquetes sobre 2,4 millones es apenas un 0,3 %, mientras que para las claves de menor frecuencia, o para las claves cuya frecuencia cambia mucho durante los 15 minutos, el mismo error superó varias veces su valor real. Por esto, la desviación estándar del error relativo es muy alta en las claves cuya frecuencia varía mucho entre ventanas. El error de CS, en cambio, tiene signo variable. En algunas ventanas CS presenta una estimación negativa, la cual se trunca a 0, lo que en esas ventanas produce un error relativo exactamente igual a 1.

La memoria de los contadores es de 45, 180 y 720 KiB por tipo de sketch para los tres anchos (nueve matrices de $d \times w$ enteros de 32 bits: las seis subventanas, el agregado y dos auxiliares para Δf), frente a los 59 MB que ocupa la tabla exacta de `exact_hh` para las IP de destino.

3.2. Detección de ataques

La Figura 2 muestra la frecuencia exacta y la estimada por CMS y CS para la clave de cada ataque, desde 60s antes del inicio hasta 60s después del término, con la semilla 1. Para ambos ataques la frecuencia presenta una forma de trapecio. Comienza a subir con el inicio del ataque y alcanza su punto más alto cuando el ataque termina (330s), ya que en ese momento todos sus paquetes están dentro de la ventana. Se mantiene en ese valor durante 30s, el ancho de la ventana

menos la duración del ataque, y después baja a medida que sus subventanas salen de la ventana. Las estimaciones siguen esta forma en los tres anchos. CMS con $w = 256$ queda visiblemente por encima de la curva exacta, tanto antes del ataque como durante este, mientras que el error de CS no se distingue a esta escala (en el Anexo se grafica el error de cada estimación).

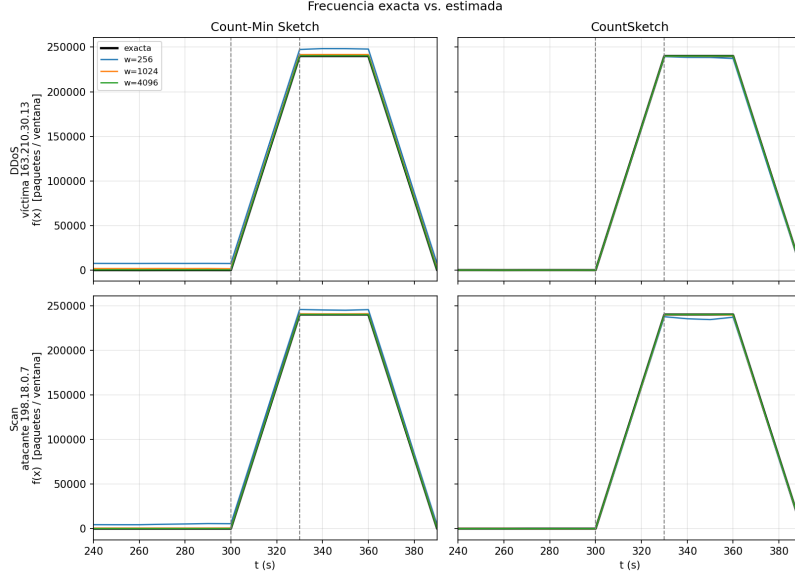


Figura 2: Frecuencia exacta y estimada de la víctima del DDoS (arriba) y del atacante del scan (abajo), con CMS (izquierda) y CS (derecha) para los tres anchos. Las líneas grises punteadas demarcan el inicio y el término del ataque.

La Tabla 1 resume la detección con las cinco semillas. Ambos ataques comienzan en $t = 300$ s, medido desde el primer paquete de la traza (parámetro `start` de `inject_attack.py`), y el conteo exacto los detecta en la evaluación de $t = 320$ s, la primera ventana en que la columna `exact_hh` de la salida de `exact_hh` vale 1, por lo que su latencia de detección es de 20s. CMS con $w = 256$ los declara una ventana antes, en $t = 310$ s, en las cinco semillas de ambos ataques. Ese adelanto es un falso positivo, producido por el sesgo positivo de CMS. CS se adelantó en una sola semilla del scan, y con $w = 1024$ y 4096 ambos sketches detectan en la misma ventana que el conteo exacto. En ningún caso hubo falsos negativos ni detecciones posteriores a la exacta. El MRE disminuye al aumentar w y es menor en CS que en CMS, salvo en el scan con $w = 4096$, donde ambos son similares.

Tabla 1: Resumen de la detección para cada ataque y ancho, con cinco semillas. MRE: media $\pm$ desviación estándar entre semillas, en las ventanas afectadas por el ataque. Falsos positivos: total de ventanas, en las cinco semillas, en que el sketch declara heavy hitter y el conteo exacto no. Detección adelantada: número de semillas, de cinco, en que el sketch detecta el ataque una ventana antes que el conteo exacto. La memoria es por tipo de sketch.

Ataque	w	Memoria (KiB)	MRE		Falsos positivos		Det. adelantada	
			CMS	CS	CMS	CS	CMS	CS
DDoS	256	45	$0,0529 \pm 0,0096$	$0,0098 \pm 0,0115$	10	1	5	0
	1024	180	$0,0110 \pm 0,0005$	$(9,9 \pm 6,3) \cdot 10^{-4}$	0	0	0	0
	4096	720	$0,0023 \pm 0,0001$	$(1,1 \pm 0,5) \cdot 10^{-4}$	0	0	0	0
Scan	256	45	$0,0305 \pm 0,0041$	$0,0205 \pm 0,0110$	10	1	5	1
	1024	180	$0,0050 \pm 0,0016$	$0,0024 \pm 0,0024$	0	0	0	0
	4096	720	$(8,8 \pm 3,1) \cdot 10^{-4}$	$(9,9 \pm 20,2) \cdot 10^{-4}$	0	0	0	0

3.3. Cambio de frecuencia

La Figura 3 muestra Δf exacto y su estimación con CS y CMS-mediana para $w = 256$, con la semilla 1. En ambos ataques, Δf exacto presenta tres incrementos mientras entran a la ventana las subventanas con paquetes del ataque, y tres decrementos de magnitud similar 60s después, cuando esas subventanas salen. Fuera de esos intervalos, Δf es cercano a 0. El mayor incremento exacto ocurre en $t = 310$ s y el mayor decremento en $t = 370$ s. Ambos estimadores siguen estos cambios.

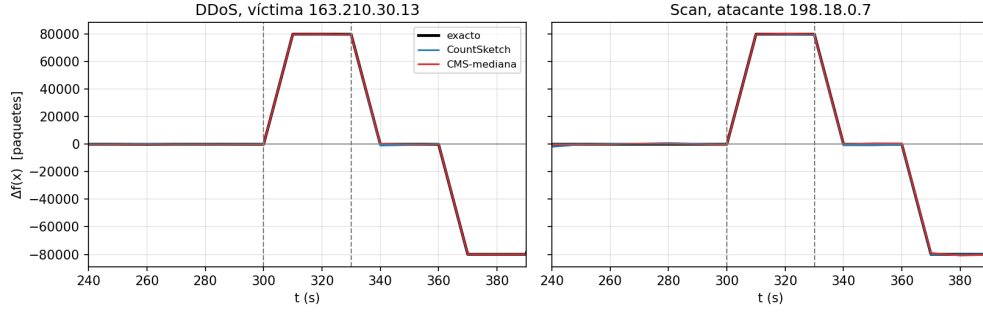


Figura 3: Cambio de frecuencia Δf exacto y estimado con CountSketch y CMS-mediana ($w = 256$) para la víctima del DDoS (izquierda) y el atacante del scan (derecha). Las líneas grises punteadas demarcan el inicio y el término del ataque.

La Tabla 2 compara el error absoluto medio de ambos estimadores respecto de Δf exacto, separando dos grupos de evaluaciones. Las de evento son las de $t = 310, 320$ y 330 s, en que entra a la ventana una subventana con paquetes del ataque, y las de $t = 370, 380$ y 390 s, en que esas subventanas salen. Las estables son todas las demás evaluaciones, desde $t = 70$ s (la primera con Δf definido) hasta $t = 890$ s. Con $w = 256$, CS tiene un error menor que CMS-mediana en las ventanas de evento de ambos ataques. Con $w = 1024$ y 4096 la diferencia es pequeña, cambia de signo según el ataque y es comparable o menor que la desviación entre semillas.

Tabla 2: Error absoluto medio (en paquetes) de la estimación de Δf respecto del valor exacto, como media $\pm$ desviación estándar entre las cinco semillas. Evento: evaluaciones de $t = 310, 320, 330, 370, 380$ y 390 s. Estables: el resto de las evaluaciones, de $t = 70$ a 890 s.

Ataque	w	Ventanas de evento		Ventanas estables	
		CS	CMS-mediana	CS	CMS-mediana
DDoS	256	$159,2 \pm 82,0$	$207,0 \pm 238,1$	$341,1 \pm 336,3$	$345,2 \pm 359,8$
	1024	$25,5 \pm 9,6$	$28,8 \pm 9,3$	$32,1 \pm 19,7$	$35,8 \pm 9,1$
	4096	$8,7 \pm 2,5$	$6,0 \pm 2,2$	$7,6 \pm 2,0$	$9,3 \pm 1,2$
Scan	256	$368,7 \pm 202,1$	$489,6 \pm 255,8$	$438,3 \pm 171,2$	$425,7 \pm 145,2$
	1024	$43,6 \pm 16,2$	$35,8 \pm 18,8$	$60,4 \pm 26,5$	$58,8 \pm 32,0$
	4096	$7,4 \pm 3,7$	$8,5 \pm 6,2$	$9,3 \pm 5,9$	$9,1 \pm 3,6$

4. Preguntas obligatorias

1. **Linealidad y costo de la ventana.** El sketch de la unión de dos flujos es la suma de sus sketches, porque cada paquete suma en celdas que dependen solo de su clave, sin importar el orden ni los demás paquetes. Por eso el agregado se actualiza con una resta y una suma de matrices (ecuación (1)), que cuestan $2dw$ operaciones por evaluación sin importar cuántos paquetes contiene la ventana, y cada paquete se inserta una sola vez. Sin linealidad habría que volver a procesar los paquetes de los últimos 60s en cada evaluación.

2. **Error de CMS y CS al reducir w .** Al reducir w aumentan las colisiones y el error de ambos crece (Tablas 1 y 3). En CMS el error es siempre positivo, porque las colisiones solo suman y el mínimo únicamente elige la fila menos afectada por ellas. Con $w = 256$ este sesgo lleva la estimación sobre el umbral una ventana antes que el conteo exacto en todas las semillas. En CS las colisiones se compensan en parte por el signo y la mediana descarta las filas más afectadas por colisiones, por lo que su error absoluto fue menor que el de CMS en todas las claves, pero puede ser positivo o negativo y varía más entre semillas (con $w = 256$ la desviación del MRE es del mismo orden que su media).
3. **Ataque detectado con mayor claridad y clave.** En la detección ambos ataques se comportan igual, ya que el conteo exacto los detecta en $t = 320$ s y los dos sketches con $w \geq 1024$ también. Solo con $w = 256$ el DDoS resulta algo más claro para CS, que no se adelantó en ningún caso y tuvo menor MRE que en el scan. La clave es distinta porque, como se describió en la Introducción, los ataques operan de manera diferente. En el DDoS muchas fuentes envían tráfico a una sola víctima, por lo que el tráfico se concentra en la IP de destino. En el scan una sola fuente contacta muchos destinos, por lo que el tráfico se concentra en la IP de origen.
4. **Información que entrega Δf .** Δf permite observar el cambio de la frecuencia entre ventanas consecutivas, es decir, el efecto de los paquetes que entran y salen de la ventana. En los ataques, permite adelantarse a la detección por HH, ya que el mayor incremento ocurre en $t = 310$ s, una evaluación antes de que el conteo exacto declare HH. Además, permite saber cuándo termina el ataque, ya que Δf vuelve a ser cercano a 0 cuando dejan de entrar paquetes del ataque ($t = 340$ s), aunque la frecuencia siga alta mientras esos paquetes permanecen en la ventana.
5. **CS frente a CMS-mediana.** En CS cada clave suma con su signo $s_j(x)$, por lo que al estimar x las colisiones quedan multiplicadas por un signo que es $+1$ o -1 con igual probabilidad. Esto se cumple tanto si las otras claves aumentan como si disminuyen su frecuencia, por lo que el error de las colisiones tiene media cero y el estimador habitual sigue siendo válido sobre ΔA . En CMS, el mínimo funciona solo porque las colisiones siempre suman y la estimación nunca es menor que el valor real. En ΔA las colisiones pueden sumar o restar, por lo que el mínimo deja de ser válido y se reemplaza por la mediana. Sin embargo, al no tener signo, el error de las colisiones no tiene media cero y se pierde la cota de error de CMS. En los resultados, CS tuvo menor error que CMS-mediana en las ventanas de evento con $w = 256$ en ambos ataques, mientras que con $w \geq 1024$ la diferencia fue comparable a la desviación entre semillas (Tabla 2).

5. Conclusiones y reproducibilidad

En esta traza, un ancho $w = 1024$, con 180 KiB por tipo de sketch frente a los 59 MB de la tabla exacta, bastó para que CMS y CS detectaran ambos ataques en la misma ventana que el conteo exacto. Con $w = 256$ el sesgo positivo de CMS adelanta la detección, mientras que CS se mantiene más cerca del valor exacto, tanto en la frecuencia como en Δf .

Se usó la traza indicada en la Introducción, convertida con `pcap2bin`. Los ataques se generaron con `inject_attack.py` con semilla 42, inicio 300 s, duración 30 s, 8.000 paquetes por segundo, 4.000 fuentes (DDoS) y 60.000 destinos (scan). Los sketches usan $d = 5$, semilla 1 en la traza sin ataques y semillas 1 a 5 con ataque. Las tablas y figuras se generan con `actividad1_codigo.py`, `generar_ataques.py`, `tabla_resumen.py`, `analisis_delta_f.py` y `Figuras.py` (comandos en README.md).

A. Anexo

Tabla 3: Error de estimación en la traza sin ataques para cinco IP de destino, en las 84 ventanas. Como se tiene un valor de error por ventana, el error absoluto (en paquetes) y el error relativo se resumen con la media $\pm$ la desviación estándar. Una desviación alta del error relativo indica que la frecuencia de esa clave varía mucho entre ventanas.

w	Clave	Rank	f exacta (media)	CMS		CS	
				Error abs.	Error rel.	Error abs.	Error rel.
256	202.12.82.146	1	2.435.327	8.655 ± 1.751	$0,004 \pm 0,001$	2.561 ± 2.246	$0,001 \pm 0,001$
	204.93.236.31	10	131.857	7.481 ± 1.003	$0,073 \pm 0,042$	684 ± 768	$0,008 \pm 0,013$
	163.210.11.14	50	2.505	7.974 ± 1.519	1.693 ± 2.629	381 ± 700	$68,9 \pm 151,4$
	163.210.52.208	100	2.512	7.390 ± 811	$28,9 \pm 46,6$	2.204 ± 1.296	$9,420 \pm 16,796$
	203.83.98.61	250	1.894	7.016 ± 810	$3,716 \pm 0,501$	1.799 ± 1.529	$0,949 \pm 0,800$
1024	202.12.82.146	1	2.435.327	1.688 ± 171	$(7,0 \pm 1,0) \cdot 10^{-4}$	168 ± 107	$(6,8 \pm 4,3) \cdot 10^{-5}$
	204.93.236.31	10	131.857	1.534 ± 87	$0,015 \pm 0,008$	129 ± 60	$0,001 \pm 0,001$
	163.210.11.14	50	2.505	1.717 ± 237	$358,2 \pm 558,4$	147 ± 172	$13,0 \pm 29,0$
	163.210.52.208	100	2.512	1.509 ± 117	$6,131 \pm 10,076$	267 ± 248	$0,760 \pm 1,349$
	203.83.98.61	250	1.894	1.583 ± 120	$0,839 \pm 0,091$	114 ± 173	$0,061 \pm 0,096$
4096	202.12.82.146	1	2.435.327	345 ± 25	$(1,4 \pm 0,2) \cdot 10^{-4}$	26 ± 16	$(1,1 \pm 0,7) \cdot 10^{-5}$
	204.93.236.31	10	131.857	346 ± 23	$0,003 \pm 0,002$	18 ± 14	$(1,7 \pm 1,4) \cdot 10^{-4}$
	163.210.11.14	50	2.505	323 ± 15	$70,5 \pm 114,5$	17 ± 16	$0,490 \pm 0,459$
	163.210.52.208	100	2.512	340 ± 20	$1,374 \pm 2,234$	25 ± 18	$0,099 \pm 0,204$
	203.83.98.61	250	1.894	330 ± 17	$0,175 \pm 0,012$	13 ± 11	$0,007 \pm 0,006$

Tabla 4: Error de estimación en la traza sin ataques para cinco IP de origen, con el mismo formato de la Tabla 3. Las claves 74.221.94.120 y 54.25.99.110 no tienen paquetes en algunas ventanas, donde el error relativo no está definido, por lo que se promedian 81 y 76 ventanas respectivamente.

w	Clave	Rank	f exacta (media)	CMS		CS	
				Error abs.	Error rel.	Error abs.	Error rel.
256	203.83.117.211	1	1.035.593	5.668 ± 1.269	$0,005 \pm 0,001$	2.334 ± 2.861	$0,002 \pm 0,003$
	74.221.94.120	10	29.093	6.290 ± 1.994	$152,7 \pm 470,3$	3.010 ± 3.348	$11,9 \pm 55,9$
	17.242.168.74	50	14.063	4.265 ± 867	$56,6 \pm 130,5$	4.093 ± 5.204	$38,8 \pm 144,4$
	173.0.177.137	100	5.572	3.606 ± 821	$2,225 \pm 1,803$	1.523 ± 1.313	$0,661 \pm 0,404$
	54.25.99.110	250	1.795	6.455 ± 2.072	$116,8 \pm 241,0$	2.497 ± 2.356	$50,3 \pm 195,1$
1024	203.83.117.211	1	1.035.593	636 ± 93	$(6,1 \pm 0,8) \cdot 10^{-4}$	113 ± 84	$(1,1 \pm 0,8) \cdot 10^{-4}$
	74.221.94.120	10	29.093	931 ± 354	$24,1 \pm 80,5$	224 ± 202	$6,500 \pm 31,118$
	17.242.168.74	50	14.063	609 ± 75	$8,905 \pm 21,287$	92 ± 126	$1,003 \pm 3,782$
	173.0.177.137	100	5.572	707 ± 103	$0,429 \pm 0,335$	149 ± 154	$0,091 \pm 0,157$
	54.25.99.110	250	1.795	830 ± 259	$13,5 \pm 28,3$	281 ± 373	$2,485 \pm 6,340$
4096	203.83.117.211	1	1.035.593	156 ± 32	$(1,5 \pm 0,3) \cdot 10^{-4}$	55 ± 61	$(5,3 \pm 5,7) \cdot 10^{-5}$
	74.221.94.120	10	29.093	125 ± 23	$3,439 \pm 10,256$	29 ± 32	$0,373 \pm 1,043$
	17.242.168.74	50	14.063	118 ± 18	$1,605 \pm 3,706$	28 ± 18	$0,263 \pm 0,639$
	173.0.177.137	100	5.572	112 ± 16	$0,067 \pm 0,052$	32 ± 45	$0,020 \pm 0,049$
	54.25.99.110	250	1.795	101 ± 12	$1,681 \pm 3,661$	65 ± 78	$0,316 \pm 0,366$

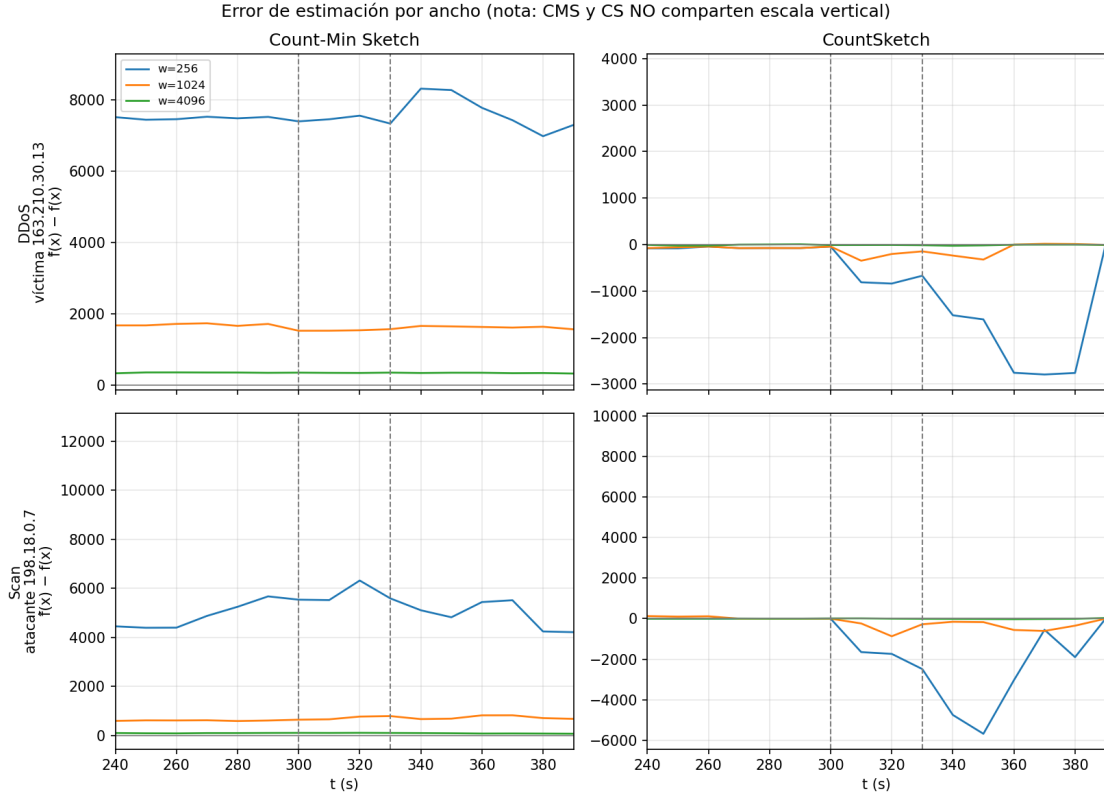


Figura 4: Error de la estimación, $\hat{f}(x) - f(x)$, para la víctima del DDoS (arriba) y el atacante del scan (abajo), con CMS (izquierda) y CS (derecha), semilla 1. CMS y CS no comparten la escala vertical. Las líneas grises punteadas demarcan el inicio y el término del ataque.